UNIVERSIDAD TECNOLÓGICA DE DURANGO

DESARROLLO DE SOFTWARE MULTIPLATAFORMA

MEMORIA
DE
ESTADÍA

“DESARROLLO DE UNA APLICACIÓN MÓVIL ANDROID DE ASISTENCIA POR VOZ PARA EL CECAPI”

POR

FERNÁNDEZ QUINTANA JOSÉ

ASESOR EXTERNO: HUGO ARMANDO PÉREZ DE LA CRUZ
ASESOR ACADÉMICO: M.T.I. OMAR ANTONIO GÓMEZ ARREOLA

GENERACIÓN: 2024-2026

VICTORIA DE DURANGO, DGO. DICIEMBRE 2026

DICTAMEN DE ESTADÍA

AGRADECIMIENTOS

RESUMEN

Las personas ciegas o con baja visión que atiende el Centro de Capacitación para Invidentes, A. C. (CECAPI), en Durango, dependen con frecuencia de otras personas para usar su teléfono, y las herramientas generales, como el lector de pantalla, exigen aprender gestos y recorrer cada elemento de la pantalla. Para atender esa necesidad se desarrolló una aplicación móvil Android de asistente por voz, con una interfaz accesible, un motor de voz único con palabra de activación, comandos globales y propios de cada pantalla, cuentas de usuario y un APK de instalación directa. El trabajo siguió un desarrollo incremental por módulos, con ramas de trabajo, revisiones de integración periódicas y decisiones de diseño documentadas, y las pantallas se recorrieron con el APK instalado en un teléfono Android. Como resultado, la aplicación cuenta con 17 destinos de navegación, una pantalla dividida con el micrófono fijo, tamaños de botón unificados, reconocimiento y síntesis de voz con tolerancia a errores, escucha en segundo plano opcional y cuentas locales con contraseñas de al menos 15 letras protegidas con Argon2id. La validación del correo con un código y las cuentas del servidor, así como la firma y la publicación del APK, forman la siguiente etapa.

Palabras clave: accesibilidad, asistente de voz, discapacidad visual, Android, aplicación móvil.

ABSTRACT

Blind and low-vision people served by the Centro de Capacitación para Invidentes, A. C. (CECAPI), in Durango, often depend on other people to use their phones, and general tools such as screen readers require learning gestures and moving through every element on the screen. To address this need, an Android voice assistant mobile application was developed, with an accessible interface, a single voice engine with a wake word, global and screen-specific commands, user accounts and a directly installable APK. The work followed an incremental, module-based development process, with working branches, periodic integration reviews and documented design decisions, and the screens were walked through with the APK installed on an Android phone. As a result, the application has 17 navigation destinations, a split screen with a fixed microphone, unified button sizes, speech recognition and synthesis tolerant to errors, optional background listening and local accounts with passwords of at least 15 letters protected with Argon2id. Email validation with a code and server accounts, as well as the signing and publication of the APK, make up the next stage.

Keywords: accessibility, voice assistant, visual impairment, Android, mobile application.

TABLA DE ABREVIATURAS

Las siglas y abreviaturas que aparecen a lo largo del documento se presentan en orden alfabético en la Tabla 1, con su significado completo y, cuando corresponde, el término original en inglés, para que el lector pueda consultar con rapidez qué significa cada una cuando la encuentre en el texto.

Tabla 1. *Siglas*

ABREVIATURA	**SIGNIFICADO**
A. C.	Asociación Civil
AOSP	Proyecto de código abierto de Android (Android Open Source Project)
API	Interfaz de programación de aplicaciones (Application Programming Interface)
APK	Paquete de aplicación de Android (Android Package)
BOM	Lista de materiales (Bill of Materials)
CECAPI	Centro de Capacitación para Invidentes, A. C.
DAO	Objeto de acceso a datos (Data Access Object)
HTTP	Protocolo de transferencia de hipertexto (Hypertext Transfer Protocol)
HTTPS	Protocolo seguro de transferencia de hipertexto (Hypertext Transfer Protocol Secure)
IA	Inteligencia artificial
INEGI	Instituto Nacional de Estadística y Geografía
NIST	Instituto Nacional de Estándares y Tecnología (National Institute of Standards and Technology)
OWASP	Proyecto Abierto de Seguridad de Aplicaciones Web (Open Worldwide Application Security Project)
SHA-256	Algoritmo de resumen seguro de 256 bits (Secure Hash Algorithm 256)
SQL	Lenguaje de consulta estructurado (Structured Query Language)
UTD	Universidad Tecnológica de Durango
WCAG	Pautas de Accesibilidad para el Contenido Web (Web Content Accessibility Guidelines)

ÍNDICE DE CONTENIDO

INTRODUCCIÓN.....XII
CAPÍTULO I GENERALIDADES DE LA EMPRESA.....13
1.1. DESCRIPCIÓN GENERAL.....13
1.2. ANTECEDENTES.....13
1.3. MISIÓN Y VISIÓN.....14
1.4. DESCRIPCIÓN GENERAL DEL PROCESO.....14
1.5. DESCRIPCIÓN DEL ÁREA DE TRABAJO.....14
CAPÍTULO II PLANTEAMIENTO DEL PROBLEMA.....16
2.1. DESCRIPCIÓN DEL PROBLEMA.....16
2.2. OBJETIVOS DEL PROYECTO.....16
2.2.1. OBJETIVO GENERAL.....16
2.2.2. OBJETIVOS ESPECÍFICOS.....17
2.3. JUSTIFICACIÓN.....17
2.4. ALCANCES Y LÍMITES.....18
2.4.1. ALCANCES.....18
2.4.2. LÍMITES.....18
2.5. VIABILIDAD.....19
CAPÍTULO III MARCO TEÓRICO.....20
3.1. ANDROID Y KOTLIN.....20
3.2. JETPACK COMPOSE Y ACCESIBILIDAD DE LA INTERFAZ.....20
3.3. DATOS LOCALES: ROOM Y DATASTORE.....21
3.4. INYECCIÓN DE DEPENDENCIAS CON HILT.....21
3.5. VOZ EN LA APLICACIÓN.....21
3.6. ACCESIBILIDAD Y USO CON VOZ.....23
3.7. REGISTRO E INICIO DE SESIÓN.....24
3.8. COMUNICACIÓN SEGURA CON EL SERVIDOR.....26
3.9. DISTRIBUCIÓN DEL APK.....26
3.10. PROTECCIÓN DE DATOS PERSONALES.....27
CAPÍTULO IV DESARROLLO DEL PROYECTO.....29
4.1. METODOLOGÍA UTILIZADA.....29
CAPÍTULO V RESULTADOS Y CONCLUSIONES.....30

5.1. RESULTADOS.....30
5.2. CONCLUSIONES.....30
5.3. RECOMENDACIONES.....30
REFERENCIAS BIBLIOGRÁFICAS.....31
ANEXOS.....34

ÍNDICE DE FIGURAS

Se completará al incluir las figuras, tablas y anexos del capítulo IV.

ÍNDICE DE TABLAS

Tabla 1. Siglas..... VI
Tabla 2. Herramientas utilizadas en el desarrollo de la aplicación.....29

ÍNDICE DE ANEXOS

Se completará al incluir las figuras, tablas y anexos del capítulo IV.

INTRODUCCIÓN

En la presente memoria se aborda el desarrollo de una aplicación móvil Android de asistente por voz para las personas ciegas o con baja visión del Centro de Capacitación para Invidentes, A. C. (CECAPI), en la ciudad de Durango. La aplicación busca que los usuarios operen su teléfono mediante la voz, con una interfaz accesible y con mayor autonomía.

El tema es relevante porque la discapacidad visual limita la autonomía cotidiana de quienes la viven y, con frecuencia, los hace depender de otras personas para actividades que otros realizan solos. En ese contexto, un teléfono que pueda manejarse por voz se convierte en una herramienta de uso diario, razón por la cual el CECAPI impulsa proyectos de inclusión tecnológica.

En el ámbito profesional, el proyecto permite aplicar los conocimientos de desarrollo de software multiplataforma a una necesidad real de una institución. En el ámbito académico, permite documentar con fuentes oficiales las decisiones técnicas sobre la voz, la accesibilidad, el registro e inicio de sesión y la distribución de la aplicación. Para su desarrollo se trabajó por módulos, con revisiones periódicas con el asesor empresarial, presenciales o por mensajería.

En el capítulo I se presentan los datos generales del CECAPI. En el capítulo II se plantea el problema, los objetivos, la justificación, los alcances y límites y la viabilidad. En el capítulo III se incluye el marco teórico sobre las tecnologías elegidas y sus condiciones de uso. En el capítulo IV se explica el desarrollo del proyecto y la metodología utilizada. En el capítulo V se presentan los resultados, las conclusiones y las recomendaciones.

CAPÍTULO I
GENERALIDADES DE LA EMPRESA

1.1. DESCRIPCIÓN GENERAL

El Centro de Capacitación para Invidentes, A. C. (CECAPI) es una asociación civil dedicada a la capacitación integral y gratuita de personas ciegas o con discapacidad visual, con sede en la ciudad de Durango. A continuación se presentan los datos generales de la institución.

Nombre de la empresa/razón social: Centro de Capacitación para Invidentes, A. C.

Dirección de la empresa: C. Jacarandá 1603, Valle Florido, código postal 34226, Durango, Dgo.

Teléfono: 618 158 7839

Correo electrónico: No cuenta con correo electrónico.

Página web: En desarrollo.

Horario: Lunes a viernes, de 15:00 a 18:30 horas.

Sector o actividad económica: Servicios (educación especial y capacitación integral sin costo para el alumno).

Servicios o productos que ofrece: Movilidad y orientación; lectoescritura en sistema Braille; matemáticas; manejo de celular y computadora; actividades deportivas y recreativas; y proyectos de inclusión tecnológica.

Área en que se realiza la estadía: Desarrollo tecnológico.

Asesor empresarial: Hugo Armando Pérez de la Cruz.

1.2. ANTECEDENTES

El CECAPI nace formalmente el 20 de marzo de 1980, a partir de un proyecto impulsado por la esposa de un gobernador del estado, quien identificó la necesidad de un espacio donde las personas con discapacidad visual pudieran adaptarse, capacitarse y desarrollar autonomía. Hoy acumula más de 46 años de trayectoria ininterrumpida.

La historia de la institución está ligada a la de su director, quien aprendió el sistema Braille de forma gratuita al final de su quinto año de primaria y, en agradecimiento, decidió

enseñarlo y difundirlo sin cobrar. Junto a la maestra Chela, especialista en la enseñanza del Braille a niños con discapacidad visual, el proyecto tomó forma de escuela.

Con los años, el CECAPI amplió su oferta a la práctica deportiva y a diversas actividades recreativas y culturales, y entre sus logros destaca contar con un egresado maratonista. En la actualidad se desarrollan dos proyectos tecnológicos: un dispositivo que detecta objetos cercanos para facilitar la movilidad y una aplicación móvil de asistencia con IA, en la que colaboran estudiantes de la UTD.

1.3. MISIÓN Y VISIÓN

Misión. Brindar capacitación integral y gratuita a personas con discapacidad visual en Durango, en áreas como movilidad, lectoescritura en sistema Braille, matemáticas, uso de tecnología y actividades deportivas y recreativas, para fortalecer su autonomía y su inclusión plena en cualquier entorno.

Visión. Ser una institución de referencia en la capacitación de personas con discapacidad visual en Durango, que amplíe de forma continua sus programas, incorpore herramientas tecnológicas adaptadas y responda a las necesidades de sus alumnos, de modo que más personas ciegas o con baja visión logren autonomía e inclusión en su entorno.

1.4. DESCRIPCIÓN GENERAL DEL PROCESO

El trabajo del CECAPI sigue un calendario escolar que inicia en septiembre y concluye a finales de junio o principios de julio, cuando los alumnos presentan un proyecto final que cierra su ciclo. Durante el año se combinan sesiones de movilidad, Braille, matemáticas, cómputo y celular con la práctica deportiva y un calendario de convivencias y celebraciones con alumnos y familias.

En los últimos años la institución ha buscado integrar la tecnología a este proceso, con el propósito de que los alumnos cuenten con herramientas adaptadas a sus necesidades para interactuar con sus propios dispositivos. Para ello impulsa proyectos de inclusión tecnológica que se prueban con los alumnos y el personal, y se ajustan a partir de sus comentarios.

1.5. DESCRIPCIÓN DEL ÁREA DE TRABAJO

El área de desarrollo tecnológico del CECAPI explora y adopta herramientas que facilitan la autonomía de los alumnos, en línea con los proyectos tecnológicos que la institución

tiene en marcha. Dentro de esta área se concentran las actividades de diseño, desarrollo y prueba de las soluciones destinadas a personas con discapacidad visual.

Los estudiantes de la UTD que realizan su estadía colaboran con esta área principalmente de manera remota, y se coordinan con el asesor empresarial para realizar pruebas con los alumnos y el personal del CECAPI cuando se requiere validar el funcionamiento de las herramientas desarrolladas.

CAPÍTULO II
PLANTEAMIENTO DEL PROBLEMA

2.1. DESCRIPCIÓN DEL PROBLEMA

Se plantea la hipótesis de que muchas personas ciegas o con baja visión dependen de otras personas para realizar actividades cotidianas con su teléfono, lo que limita su autonomía. Frente a esta dependencia, los alumnos del CECAPI no cuentan con una aplicación propia, adaptada a sus necesidades, que concentre sus funciones y que pueda manejarse mediante la voz.

Se plantea también que, aun cuando los teléfonos incluyen lectores de pantalla y existen aplicaciones como Voice Access, que “lets you control your device with spoken commands” (Google, s. f.-b, sección Get started with Voice Access spoken commands, párr. 1), no están pensadas específicamente para los alumnos del CECAPI ni reúnen en un solo lugar las funciones que necesitan. Se considera, además, que el manejo del lector de pantalla puede resultar lento para quienes se inician en el celular, pues requiere aprender gestos y recorrer los elementos de la pantalla; una aplicación con comandos de voz propios busca reducir esos pasos.

Los destinatarios de la aplicación son las personas adultas del CECAPI. La institución atiende también a menores, pero ellos no tienen acceso a la aplicación ni a la inteligencia artificial y realizan otras actividades.

Por ello se requiere una aplicación con navegación y asistente de voz, con cuentas que permitan conservar las conversaciones con el asistente y los ajustes de voz de cada alumno, y con una distribución que pueda instalarse directamente en sus teléfonos, sin depender de una tienda de aplicaciones.

2.2. OBJETIVOS DEL PROYECTO

2.2.1. OBJETIVO GENERAL

Desarrollar una aplicación móvil Android de asistente por voz, con cuentas de usuario y un APK de instalación directa, mediante una interfaz accesible y una navegación por voz, con el fin de apoyar a las personas con discapacidad visual del CECAPI en sus tareas cotidianas y permitirles usar su teléfono con mayor autonomía.

2.2.2. OBJETIVOS ESPECÍFICOS

- Diseñar las pantallas de la aplicación con etiquetas descriptivas para el lector de pantalla y con señales distintas del color, para que los usuarios ciegos las recorran con el lector de pantalla y los de baja visión distingan sus elementos.
- Programar un asistente de voz que, con el consentimiento del usuario para la escucha del micrófono, se active con una palabra de activación, reconozca comandos globales y propios de cada pantalla de la aplicación, que envíe al servidor las preguntas del usuario y lea en voz alta la respuesta de la inteligencia artificial, y que avise cuando no entiende, mediante los servicios de reconocimiento y síntesis de voz de Android, para que los usuarios controlen con su voz las funciones de la aplicación y consulten al asistente sin usar la pantalla.
- Construir las pantallas de registro e inicio de sesión, con una sola forma de ingreso (una cuenta propia con una contraseña pensada para el uso por voz, protegida con Argon2id), y dejar preparada la validación del correo electrónico con un código enviado por el servidor central, que también se usará para recuperar la contraseña, para identificar a cada usuario y conservar sus conversaciones.
- Distribuir la aplicación como archivo APK de instalación directa en teléfonos Android, para que los alumnos del CECAPI puedan instalarla sin depender de una tienda de aplicaciones.

2.3. JUSTIFICACIÓN

La relevancia social del proyecto se sustenta en los datos del Instituto Nacional de Estadística y Geografía [INEGI] (2024, pp. 1-3): en 2023 había en el país 8.8 millones de personas de 5 años y más con alguna discapacidad, es decir, con mucha dificultad o sin poder realizar al menos una actividad, y 45.8 % de ellas tenía dificultad para ver aun usando lentes. Durango ocupa el tercer lugar entre los estados con mayor proporción de población con discapacidad (9.9 %). En este contexto, ofrecer a los alumnos del CECAPI una herramienta propia les permite aprovechar la tecnología con mayor autonomía.

El proyecto es conveniente porque reúne en una sola aplicación, con navegación por voz, las funciones que los alumnos necesitan en su teléfono, en lugar de depender de aplicaciones separadas con reglas de accesibilidad distintas. En la práctica, la institución cuenta con una aplicación adaptada que puede probarse con sus alumnos y ajustarse a partir de sus comentarios.

En el aspecto teórico, el capítulo III documenta con fuentes oficiales las decisiones sobre la voz, la accesibilidad, el registro e inicio de sesión y la distribución del APK, que pueden servir de referencia a proyectos semejantes. En el aspecto metodológico, se documenta el criterio de contraseña adoptado para el uso por voz, su contraste con la norma del NIST y las medidas que lo compensan, de modo que el procedimiento quede descrito para su revisión. Como consecuencia, se espera que los usuarios utilicen su teléfono con menor dependencia de otras personas, que es el propósito que orienta el diseño de la aplicación.

2.4. ALCANCES Y LÍMITES

2.4.1. ALCANCES

La aplicación incluye interfaz accesible, asistente de voz con palabra de activación (el nombre del asistente, que la persona puede personalizar, y la palabra hola si se activa) para el control de la aplicación y para hacer preguntas, cuyas respuestas genera la inteligencia artificial del servidor central, pantallas de registro e inicio de sesión con una sola forma de ingreso (cuenta propia con una contraseña pensada para el uso por voz, preparada para validar el correo con un código que enviará el servidor central), confirmación de mayoría de edad con su fecha de registro, y distribución en APK para teléfonos Android. Las funciones externas a la aplicación, como el volumen, el brillo, los gestos y los movimientos del teléfono, corresponden al módulo de accesibilidad y gestos, desarrollado por otros estudiantes. La voz de la interfaz y de las respuestas del asistente corresponde a esta aplicación, y el módulo de lectura reutiliza el mismo motor de síntesis de voz. La aplicación se integra con el servidor central, que atiende las cuentas y la inteligencia artificial, y con tres módulos desarrollados por otros estudiantes: entorno y accesibilidad, lectura de documentos y aprendizaje. El funcionamiento de la aplicación se comprueba durante el desarrollo mediante recorridos de prueba en el teléfono.

2.4.2. LÍMITES

Los términos de uso de la API del modelo de inteligencia artificial exigen tener al menos 18 años y no permiten usarla en un cliente dirigido a menores de 18 años o que probablemente sea accedido por ellos (Google, 2026, sección Age Requirements). Por ello la aplicación está destinada únicamente a personas mayores de edad y se solicita una declaración de mayoría de edad al registrarse, que no sustituye una verificación de la edad. Los menores que asisten al CECAPI no usan la aplicación; sin embargo, como el APK se distribuye desde GitHub Releases, ninguna medida técnica impide que un menor lo instale, por lo que el control es declarativo y se

reconoce como un riesgo. Como medida para reducirlo se considera que el CECAPI entregue el enlace de descarga solo a sus alumnos adultos inscritos; su adopción depende de la institución.

No se desarrolla una versión para iPhone, y la distribución se realiza mediante APK, sin publicación en tiendas de aplicaciones durante la estadía; esta modalidad exige habilitar la instalación desde fuentes desconocidas y no recibe actualizaciones automáticas.

El reconocimiento de voz depende de que el teléfono tenga descargado el paquete de voz para funcionar sin internet. Las funciones disponibles dependen de la versión de Android de cada teléfono, y lo que no se concluya al cierre queda documentado como trabajo futuro.

El proyecto se desarrolla de septiembre a diciembre de 2026. El CECAPI cubre el servidor y se contempla que el dominio lo cubran el CECAPI y los estudiantes. Por ahora las consultas a la inteligencia artificial se realizan con las condiciones de los servicios sin pago, y el paso a un servicio de pago se definirá más adelante.

La validación del correo con un código depende de un servicio de correo por interfaz web, de un dominio propio y de que el servidor central publique las rutas de cuenta, y el funcionamiento completo depende de que el servidor central y los módulos de otros estudiantes estén disponibles para su integración.

2.5. VIABILIDAD

En los meses previos a la estadía se desarrollaron prototipos de la aplicación, con funciones que operaban pero no estaban finalizadas, y esa fue la base del desarrollo del proyecto. En lo técnico, la aplicación se programa para Android y se apoya en el servidor central. En lo operativo, el trabajo se realiza principalmente de forma remota y se coordina con el asesor empresarial para las pruebas con los alumnos. El principal riesgo es la carga de un equipo pequeño, que se reduce porque las partes de la aplicación se desarrollan por separado y se realizan revisiones de integración periódicas, cuyo resultado se registra en el repositorio del proyecto para que el avance pueda verificarse.

CAPÍTULO III
MARCO TEÓRICO

El marco teórico reúne las fuentes que brindan soporte al proyecto: consiste en localizar y revisar la literatura relacionada, y en integrar la información necesaria para sustentar y documentar la investigación (Universidad Tecnológica de Durango [UTD], s. f.). En este capítulo se presentan únicamente los fundamentos de lo que se desarrolla en la aplicación móvil: la plataforma, la interfaz, los datos locales, la voz, la accesibilidad, el registro e inicio de sesión, la comunicación segura con el servidor, la distribución del APK y la protección de datos personales. No se presenta como un glosario: en cada tema se explica en qué consiste la tecnología, por qué se eligió y qué limitaciones tiene. La información proviene principalmente de la documentación oficial de cada tecnología, consultada entre el 26 y el 29 de octubre de 2026. Las citas textuales de fuentes en inglés se presentan en su idioma original, conforme al manual de la universidad (Universidad Tecnológica de Durango, s. f.).

3.1. ANDROID Y KOTLIN

La aplicación se desarrolla para Android, el sistema operativo de los teléfonos que utilizan los alumnos del CECAPI, con el nivel de API 26 como versión mínima compatible y el nivel de API 35 como versión objetivo. El lenguaje elegido es Kotlin, que cuenta con soporte completo para el desarrollo en Android y con soporte de primera clase en Android Studio, su entorno de desarrollo (Android Developers, 2026f).

Kotlin se justifica además porque la propia plataforma lo adopta: el kit de herramientas moderno para crear interfaces de Android está construido sobre este lenguaje, que permite escribir menos código repetitivo (Android Developers, 2026f). En un proyecto con plazo de cuatro meses, un lenguaje conciso reduce el código que se debe escribir, revisar y probar.

3.2. JETPACK COMPOSE Y ACCESIBILIDAD DE LA INTERFAZ

La interfaz se construye con Jetpack Compose, descrito por Android como “the modern toolkit for building Android UI, simplifying the development of apps that adapt to any display size” (Android Developers, 2026e, sección Get started with Jetpack Compose, párr. 1). Su enfoque es declarativo, distinto del basado en vistas que se utilizaba antes (Android Developers, 2026e): la pantalla se describe a partir del estado de la aplicación y se actualiza cuando ese estado cambia.

En una aplicación para personas ciegas o con baja visión, cada elemento interactivo necesita una etiqueta descriptiva, porque “screen readers such as TalkBack can announce these labels to users” (Android Developers, 2026h, sección Label elements, párr. 1). Además, conviene seguir la indicación de “use cues other than color to distinguish UI elements within your app's screens” (Android Developers, 2026h, sección Use cues other than color, párr. 1), lo cual favorece a los usuarios con baja visión que no distinguen bien los colores.

3.3. DATOS LOCALES: ROOM Y DATASTORE

Los datos que la aplicación conserva en el teléfono se almacenan con Room, una biblioteca que proporciona una capa de abstracción sobre SQLite y se organiza en tres componentes: la base de datos, las entidades que representan las tablas y los objetos de acceso a datos (DAO), que consultan, insertan, actualizan y eliminan la información. Entre sus ventajas destacan la verificación de las consultas SQL durante la compilación y las rutas de migración simplificadas (Android Developers, 2026j).

Los ajustes sencillos del usuario, como la velocidad de la voz, se guardan con DataStore, que almacena pares clave-valor u objetos tipados de forma asíncrona, consistente y transaccional mediante corrutinas de Kotlin y Flow. Para conjuntos de datos grandes o complejos conviene Room en lugar de DataStore (Android Developers, 2026b), por lo que cada herramienta se destina al tipo de información que le corresponde.

3.4. INYECCIÓN DE DEPENDENCIAS CON HILT

Hilt es la biblioteca de inyección de dependencias recomendada oficialmente para Android, y reduce el código repetitivo que implica realizar esa inyección de forma manual (Android Developers, 2026c). La inyección de dependencias consiste en que cada componente reciba desde fuera los objetos que necesita, en lugar de crearlos por su cuenta. Como la aplicación se compone de varios módulos desarrollados por distintos estudiantes, este esquema evita crear a mano las dependencias de cada módulo y facilita sustituirlas durante las pruebas.

3.5. VOZ EN LA APLICACIÓN

La aplicación se maneja principalmente por voz y para ello utiliza dos componentes de Android. SpeechRecognizer proporciona acceso al servicio de reconocimiento de voz y convierte lo que dice el usuario en texto; su documentación advierte que la implementación de esta interfaz probablemente transmite el audio a servidores remotos y que no está pensada para el

reconocimiento continuo, por el consumo de batería y de datos que implica (Android Open Source Project [AOSP], s. f.-b). TextToSpeech, por su parte, sintetiza voz a partir de texto para reproducirla de inmediato o crear un archivo de sonido, y solo puede usarse una vez completada su inicialización (AOSP, s. f.-c).

El funcionamiento sin conexión es una limitación que debe considerarse. La opción para preferir el reconocimiento sin conexión tiene como valor predeterminado falso, lo que significa que pueden usarse motores con red o sin ella, y según el reconocedor instalado puede incluso no tener efecto (AOSP, s. f.-a). En el código de la aplicación el reconocimiento se solicita sin esa opción, por lo que solo funciona sin internet si el teléfono tiene descargado el paquete de voz correspondiente. Por esta razón, las funciones de voz sin internet dependen de ese paquete y no se garantizan en todos los teléfonos.

La palabra de activación mantiene la aplicación atenta sin que el usuario toque la pantalla; puede ser el nombre de la aplicación o un nombre que el usuario personaliza. Como SpeechRecognizer no está pensado para el reconocimiento continuo (AOSP, s. f.-b), la aplicación solicita reconocimientos sucesivos y, cuando falla la red, espera cada vez más tiempo antes de reiniciar la escucha. La finalidad de mantener la escucha fuera de la aplicación es que el usuario pueda consultar al asistente sin volver a abrirla; el control de otras aplicaciones y de las funciones del teléfono, como el volumen o el brillo, requiere un servicio de accesibilidad y corresponde al módulo de accesibilidad y gestos.

Para escuchar en segundo plano, Android exige un servicio en primer plano con tipo declarado: “Beginning with Android 14 (API level 34), you must declare an appropriate service type for each foreground service” (Android Developers, 2026d, sección Foreground service types, párr. 1); para el micrófono se requieren el permiso correspondiente de servicio en primer plano y el permiso de grabación de audio (Android Developers, 2026d, sección Microphone, viñetas de requisitos). Esta misma sección establece una restricción que condiciona el diseño: “The RECORD_AUDIO runtime permission is subject to while-in-use restrictions. For this reason, you cannot create a microphone foreground service while your app is in the background” (Android Developers, 2026d, sección Microphone, nota). Por tanto, la escucha solo puede iniciarse cuando la aplicación está visible, por ejemplo al iniciar sesión, y no puede reiniciarse por sí sola tras reiniciar el teléfono o cuando el sistema detiene el servicio. La escucha se mantiene mientras el usuario tiene una sesión iniciada y se detiene al cerrar la sesión o la aplicación; si el sistema la interrumpe, el usuario debe abrir la aplicación para reanudarla, situación para la cual el servicio mantiene una notificación visible mientras está activo y, al

detenerse, se prevé que la aplicación avise por voz en cuanto vuelva a abrirse, lo cual se probará en un teléfono real.

Como el reconocimiento de voz probablemente transmite el audio a servidores remotos (AOSP, s. f.-b), lo que se diga cerca del teléfono mientras la escucha está activa puede salir del dispositivo, y esto alcanza también a personas cercanas que no son usuarias. Tener una cuenta no reduce esa transmisión; la limitación a usuarios con sesión iniciada solo evita que la escucha funcione sin que alguien haya accedido a la aplicación. La autorización del CECAPI corresponde a la realización del proyecto y no sustituye el consentimiento de cada alumno, por lo que se prevé un aviso hablado de esta condición, un botón explícito de encendido y apagado, y la aceptación del usuario antes de activar la escucha en segundo plano, que se realiza mediante un botón y no solo dictando «acepto», porque lo dictado también pasa por el reconocimiento en línea. El comando de voz para apagarla también pasa por el reconocimiento en línea mientras la escucha está activa, por lo que el botón es la vía principal. Queda por decidir si la palabra de activación se reconoce de forma local, sin enviar audio, decisión que incide en la privacidad y en el consumo de batería. Como la aplicación trata datos personales, como el correo y las conversaciones, se toma como marco de referencia la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (Ley Federal de Protección de Datos Personales en Posesión de los Particulares, 2025), por lo que se prevé presentar un aviso de privacidad que se lee en pantalla y por audio.

3.6. ACCESIBILIDAD Y USO CON VOZ

La accesibilidad es el propósito central de la aplicación. Las pautas WCAG 2.2 (Pautas de Accesibilidad para el Contenido Web) fueron publicadas por el World Wide Web Consortium, y el documento afirma que “defines how to make web content more accessible to people with disabilities” (World Wide Web Consortium, 2024, sección Background on WCAG 2, párr. 1). Se basan en cuatro principios: el contenido debe ser perceptible, operable, comprensible y robusto (World Wide Web Consortium, 2024, sección WCAG 2 Layers of Guidance). Aunque están redactadas para la web, sirven como orientación general para la aplicación.

Android parte del mismo principio: “people with disabilities depend on apps and services that support accessibility to communicate, learn, and work” (Android Developers, 2026a, sección Increase your app’s reach, párr. 1), y por ello recomienda “succinctly describing each UI element and ensuring all user flows are accessible” (Android Developers, 2026a, sección Develop, párr. 1). Para una persona ciega como usuario principal, esto implica que cada botón

tenga una descripción clara para el lector de pantalla y que ninguna tarea dependa de ver la pantalla.

Entre las soluciones existentes, Voice Access de Google permite controlar el teléfono por voz: “The Voice Access app for Android lets you control your device with spoken commands” (Google, s. f.-b, sección Get started with Voice Access spoken commands, párr. 1). La aplicación del CECAPI no sustituye a esa herramienta general; ofrece un asistente con comandos y respuestas habladas propios de cada pantalla, y que puede mantenerse a la escucha fuera de la aplicación, de modo que el usuario pueda consultarlo sin tocar la pantalla; el control del resto del teléfono queda a cargo del módulo de accesibilidad y gestos. Android cuenta además con la aplicación Gemini, que permite hacer preguntas por voz y que, al elegirla, sustituye a Google Assistant como asistente principal del dispositivo (Google, s. f.-a). Voice Access y Gemini son herramientas generales de Google; el asistente del proyecto no pretende sustituirlas, sino reunir en una aplicación propia, con comandos por pantalla y respuestas habladas, las funciones definidas con el CECAPI para sus alumnos adultos.

En cuanto al reconocimiento de voz, el audio puede salir del teléfono hacia servidores remotos (véase la sección 3.5). La contraseña puede dictarse o escribirse con el teclado, y se espera que lo más frecuente sea dictarla; hacerlo en voz alta, sobre todo en lugares públicos, representa un riesgo de privacidad para el usuario, ya que otras personas cercanas pueden escucharla, por lo que la escritura queda disponible como alternativa. Además, al dictarla, la contraseña viaja como audio al servicio de reconocimiento antes de convertirse en texto, tramo que no queda cubierto por el envío por HTTPS ni por el almacenamiento con sal descritos en las secciones 3.7 y 3.8, los cuales protegen únicamente el texto que llega al servidor de la aplicación.

3.7. REGISTRO E INICIO DE SESIÓN

El ingreso a la aplicación se realiza con una sola forma: una cuenta propia, a la que se entra con un nombre de usuario y una contraseña. El correo electrónico es opcional; cuando se proporciona, debe ser un correo existente y se valida con un código que el servidor envía a esa dirección, y el mismo procedimiento se usa para recuperar la contraseña. No se contempla el inicio de sesión con Google. Esta sección describe el diseño de la cuenta propia, que se programa en la aplicación. Las pantallas de registro e inicio de sesión se diseñaron considerando el uso de la voz. La contraseña se compone únicamente de letras en minúsculas, con un mínimo de 15 letras, y no admite espacios, símbolos ni números, porque estos caracteres pueden no reconocerse con fiabilidad al dictar; la ñ cuenta como letra. Como el reconocimiento devuelve las palabras separadas por espacios y con acentos, la aplicación debe eliminar los espacios,

quitar el acento y la diéresis a las vocales (la ñ se conserva) y convertir el texto a minúsculas antes de enviarlo por HTTPS al servidor, que repite la misma normalización antes de guardarlo o compararlo (la aplicación ya aplica esta normalización y el servidor deberá repetirla); así, una frase dictada queda como una sola cadena de letras que coincide con lo registrado. Quitar el acento hace iguales palabras como «mamá» y «mama», lo que reduce un poco la variedad posible de contraseñas pero evita rechazos por la forma en que el reconocimiento escribe las palabras. El NIST (Instituto Nacional de Estándares y Tecnología) establece en cambio que no deben imponerse reglas de composición: “Other composition requirements for passwords SHALL NOT be imposed” (National Institute of Standards and Technology [NIST], 2025, sección 3.1.1.1), y sugiere aceptar el carácter de espacio y los caracteres Unicode (NIST, 2025, sección 3.1.1.2). La regla del proyecto se aparta de esa norma por la limitación de la voz, y lo compensa con el largo mínimo de 15 caracteres que la misma norma fija para una contraseña usada como único factor de autenticación (NIST, 2025, sección 3.1.1.2). Las demás medidas corresponden al servidor central, que desarrolla otro equipo y que aún se encuentra en desarrollo, por lo que se presentan como el diseño previsto: guardar la contraseña con Argon2id, variante que OWASP recomienda con una configuración mínima de 19 MiB de memoria, dos iteraciones y un grado de paralelismo (OWASP Cheat Sheet Series, s. f.-b, sección Argon2id), y que NIST exige con sal (NIST, 2025, sección 3.1.1.2), limitar los intentos fallidos consecutivos (NIST, 2025, sección 3.2.2) y rechazar contraseñas comunes o filtradas (NIST, 2025, sección 3.1.1.2).

Para comprobar que el correo electrónico pertenece al usuario se prevé enviar un código de seis dígitos que debe ingresarse en la aplicación, procedimiento que coincide con la recomendación de OWASP: “Users should be permitted to use their email address as a username, provided the email is verified during sign-up” (OWASP Cheat Sheet Series, s. f.-a, sección Usernames, párr. 2). El envío y la validación del código corresponden al servidor; la aplicación se encarga de solicitarlo y de recibirlo por voz o por teclado. La recuperación de la contraseña sigue el mismo procedimiento: el usuario solicita un código a su correo y define una contraseña nueva, que pasa por la misma normalización. El tiempo de vigencia del código y el número de intentos permitidos los define el servidor y están por definirse. El envío requiere un servicio de correo por interfaz web y un dominio propio verificado, pues el que ofrece la base de datos es solo para pruebas; el dominio y el remitente se solicitarán al CECAPI y están por definir, por lo que esta función solo opera plenamente cuando el servidor cuente con ellos.

3.8. COMUNICACIÓN SEGURA CON EL SERVIDOR

Toda la información que la aplicación intercambia con el servidor central (cuentas, códigos, contraseñas y conversaciones) viaja por internet, por lo que su protección en tránsito es un requisito del diseño. La comunicación entre ambos debe realizarse cifrada mediante HTTPS (protocolo seguro de transferencia de hipertexto). A partir de Android 9 (nivel de API 28), el soporte para tráfico sin cifrar viene desactivado por defecto (Android Developers, 2026g, sección Cleartext traffic). La aplicación debe utilizar únicamente direcciones que comiencen con https:// y no activar la excepción para tráfico sin cifrar, una configuración insegura que debe evitarse siempre que sea posible (Android Developers, 2026g, sección Opt in to cleartext traffic).

3.9. DISTRIBUCIÓN DEL APK

La aplicación se distribuye como APK (paquete de aplicación de Android), de instalación directa. Android exige la firma digital: “Android requires that all APKs be digitally signed with a certificate before they are installed on a device or updated” (Android Developers, 2026k, sección Sign your app, párr. 1). La clave privada es necesaria para firmar todas las versiones futuras y, si se pierde, “you will not be able to publish updates to your existing app” (Android Developers, 2026k, sección Keep your key secure, párr. 4), por lo que la llave de firma se conserva con respaldo y fuera del repositorio.

El código del proyecto se almacena en GitHub, y desde ahí se distribuye la aplicación. Cada versión del APK se publica como una release, descrita como “deployable software iterations you can package and make available for a wider audience to download and use” (GitHub, 2026, sección About releases, párr. 1). Los límites de la plataforma son adecuados para este uso: cada archivo debe pesar menos de 2 GiB y “there is no limit on the total size of a release, nor bandwidth usage” (GitHub, 2026, sección Storage and bandwidth quotas, párr. 1). Por el momento se descartó la publicación en tiendas de aplicaciones, por lo que cada versión nueva se instala de forma manual, y el botón de descarga del sitio web de la aplicación, desarrollado por otro equipo, enlaza a la release. Además, desde el 30 de septiembre de 2026 Google exige el registro de las aplicaciones en las tiendas participantes de Brasil, Indonesia, Singapur y Tailandia, y prevé extender la medida a todas las aplicaciones de los dispositivos certificados en 2027 (Forsythe, 2026); México no figura en la primera etapa. La instalación de aplicaciones de desarrolladores no registrados seguirá siendo posible mediante un flujo avanzado (Forsythe, 2026), y para estudiantes existe una cuenta de distribución limitada,

gratuita y sin identificación oficial, que permite compartir aplicaciones con hasta 20 dispositivos autorizados por sus usuarios (Android Developers, 2026i, sección What can a limited distribution account do?).

3.10. PROTECCIÓN DE DATOS PERSONALES

La aplicación trata datos personales: el nombre de usuario, el nombre, el rol, la fecha en que la persona confirmó tener 18 años o más, el correo electrónico (opcional), el resumen cifrado de la contraseña y las conversaciones con el asistente. La aplicación no solicita datos de salud y se advierte al usuario que no los dicte; sin embargo, saber que una persona usa una aplicación para personas ciegas puede revelar su discapacidad visual, que el proyecto trata como dato sensible (decisión 0018). Como marco de referencia se toma la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (2025). Según su texto, quien trata datos debe poner a disposición del titular un aviso de privacidad que indique, entre otros puntos, la identidad y el domicilio del responsable, los datos tratados, las finalidades y los medios para ejercer los derechos de acceso, rectificación, cancelación y oposición (art. 15); debe obtener el consentimiento del titular (art. 7); debe suprimir los datos que ya no necesite (art. 10); y debe mantener medidas de seguridad administrativas, técnicas y físicas (art. 18) (Ley Federal de Protección de Datos Personales en Posesión de los Particulares, 2025). Para los datos personales sensibles, el responsable debe obtener el consentimiento expreso y por escrito de la persona titular, mediante firma autógrafa, firma electrónica o cualquier mecanismo de autenticación (art. 8) (Ley Federal de Protección de Datos Personales en Posesión de los Particulares, 2025). Por ello, el mecanismo con que se acepte el aviso de privacidad se someterá a la revisión de una persona abogada antes de usar datos reales (decisión 0005).

En el proyecto, el aviso de privacidad se presentará en la aplicación, se leerá en pantalla y por audio, y se aceptará con un botón. Se prevé que las cuentas y las conversaciones las guarde el servidor central, desarrollado por otro equipo, al que corresponde la supresión de datos, la atención de los derechos de los titulares y las medidas de seguridad, que se describen como diseño previsto. Por la decisión 0005, mientras solo haya cuentas de demostración el responsable del tratamiento es el responsable del proyecto, a título personal, y antes de usar datos reales se buscará un convenio con el CECAPI, A. C. Se prevé que el aviso se lea y se acepte antes de la primera pregunta a la inteligencia artificial. El servidor se aloja en Render y la base de datos en Neon, que procesan datos principalmente en Estados Unidos (Render, s. f.-a; Neon, s. f.); Render prohíbe además almacenar datos sensibles, como los de salud, y el uso por menores de 16 años

(Render, s. f.-b). Las tablas de usuarios de la aplicación no incluyen la discapacidad visual como campo.

El servicio de inteligencia artificial que usa el servidor se apoya en las API de Gemini y de Groq. Por ahora se utiliza sin pago, por lo que aplican las condiciones de los servicios sin pago de Gemini: Google usa el contenido enviado y las respuestas generadas para proporcionar, mejorar y desarrollar sus productos, revisores humanos pueden leer, anotar y procesar las entradas y salidas, y se pide no enviar información sensible, confidencial o personal (Google, 2026, sección Unpaid Services). Si más adelante se contrata un servicio de pago, los términos indican que Google no usa los mensajes ni las respuestas para mejorar sus productos, aunque los registra durante un tiempo limitado para detectar usos prohibidos (Google, 2026, sección Paid Services), condición que tendría que confirmarse al activar la facturación.

Groq, el otro proveedor, indica que por defecto no conserva los datos de las solicitudes de inferencia, salvo registros para investigar abusos que se guardan hasta 30 días (Groq, s. f.); su documentación consultada no precisa si usa esos datos para entrenar sus modelos.

Como las preguntas de los usuarios pueden contener datos personales o de salud, se prevén tres medidas: informarlo en el aviso de privacidad y por voz antes del primer uso, indicar al usuario que no dicte datos personales ni de salud al asistente, y aplicar en el servidor un filtro de privacidad que cubre correo, teléfono, tarjeta, CURP, RFC y contraseñas, pero no cubre nombres, domicilios ni datos de salud.

CAPÍTULO IV
DESARROLLO DEL PROYECTO

4.1. METODOLOGÍA UTILIZADA

Las herramientas utilizadas en el desarrollo de la aplicación se resumen en la Tabla 2, que indica el nombre de cada una, su versión y la función que cumple dentro de la aplicación. Las versiones corresponden a las declaradas en el proyecto al 2 de noviembre de 2026. Las que utilizan los demás módulos, como la cámara y el servidor, se describen en las memorias correspondientes.

Tabla 2. *Herramientas utilizadas en el desarrollo de la aplicación*

Herramienta	**Versión**	**Uso en la aplicación**
Kotlin	2.0.21	Lenguaje de programación
Android Gradle Plugin	8.5.2	Compilación del proyecto
Gradle	8.9	Sistema de compilación
Jetpack Compose (BOM)	2024.10.00	Interfaz de usuario
Navigation Compose	2.8.2	Navegación entre pantallas
Lifecycle (ViewModel)	2.8.6	Estado de cada pantalla
Coroutines	1.9.0	Tareas en segundo plano
Room	2.6.1	Base de datos local
DataStore	1.1.1	Ajustes del usuario
Hilt	2.51.1	Inyección de dependencias
Argon2kt	1.5.0	Protección de contraseñas con Argon2id
Cliente de red (HTTPS)	Incluido en Android	Comunicación con el servidor central
Servicio en primer plano	Incluido en Android	Escucha con la palabra de activación
SpeechRecognizer	Incluido en Android	Reconocimiento de voz
TextToSpeech	Incluido en Android	Respuestas habladas
Git y GitHub	No aplica	Control de versiones del código

El proyecto es un desarrollo de software aplicado a una necesidad de la institución, y su marco de trabajo fue incremental por módulos independientes: cada módulo se construyó, se integró y se ajustó por separado, con una rama de trabajo por tarea, revisiones de integración periódicas y decisiones de diseño escritas en documentos numerados que explican el contexto, lo

decidido y sus consecuencias. Cada versión de la aplicación se compiló como APK de instalación directa y se instaló en un teléfono Android para recorrer sus pantallas.

4.1.1. ORGANIZACIÓN DEL TRABAJO

La aplicación se divide en siete módulos, cada uno en su propia carpeta dentro de la carpeta feature del proyecto, y una carpeta compartida llamada core. Esta memoria describe el módulo 1, que reúne el inicio, el menú, las cuentas, los ajustes, la personalización y los chats, y el módulo 2, el asistente de voz, además del motor de voz, la base de datos y el servicio de escucha de la carpeta compartida. Los demás módulos los desarrollan otros estudiantes y se describen en las memorias correspondientes; aquí solo se citan como dependencias.

La Tabla 3 resume el tamaño del código descrito en esta memoria, contado en líneas no vacías con los comentarios incluidos, y la Figura 1 muestra los cambios registrados por semana en esas carpetas hasta el 2 de noviembre de 2026. La mayor actividad se registró en la semana del 28 de septiembre al 4 de octubre, con 30 cambios, y en la del 26 de octubre al 1 de noviembre, con 21.

Tabla 3. *Código de los módulos 1 y 2 y su apoyo en core*

Parte	Carpeta	Archivos	Líneas
Módulo 1. Aplicación principal	feature/modulo1_aplicacionprincipal	35	8,225
Módulo 2. Asistente de voz (pantalla y datos)	feature/modulo2_asistentevoz	4	297
Motor y comandos de voz del asistente	core/voice	18	1,711
Base de datos y migraciones	core/data	3	356
Servicio de escucha en segundo plano	service	1	159

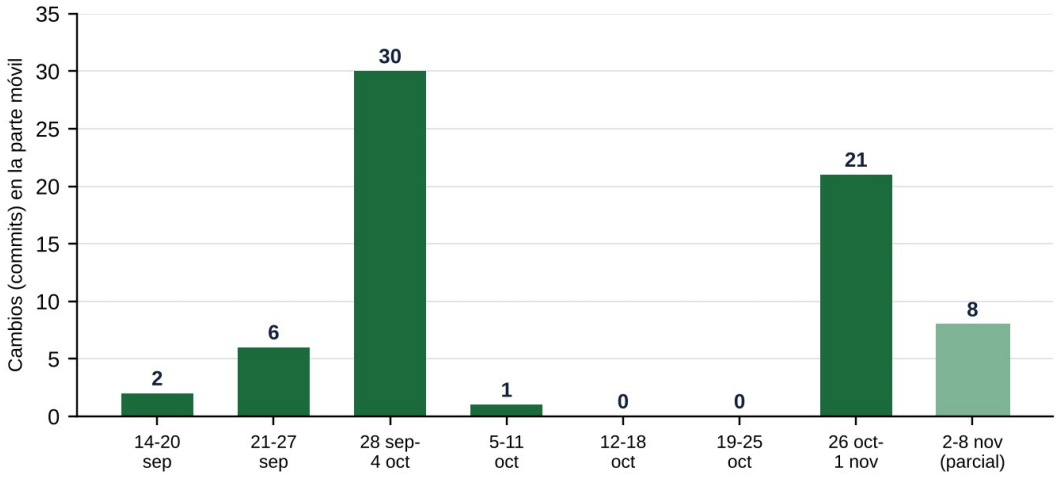


Figura 1. *Cambios registrados por semana en las carpetas de los módulos 1 y 2*

Las reglas de colaboración quedaron escritas en una guía del repositorio: cada equipo edita solo la carpeta de su módulo, lo compartido vive en core, y cada cambio llega con una descripción de lo que modifica y de cómo se probó. Las versiones de las herramientas de compilación se congelaron desde el inicio (decisión 0001), de modo que Gradle 8.9, el complemento de Android 8.5.2 y Kotlin 2.0.21 no cambian sin acuerdo, porque una actualización a versiones más nuevas ya había roto la compilación desde un clon limpio.

4.1.2. TÉCNICAS E INSTRUMENTOS DE RECOLECCIÓN DE DATOS

Los datos del proyecto se recolectaron con los instrumentos de la Tabla 4. El historial del repositorio permitió medir el avance por semana, las decisiones numeradas, reunidas en el Anexo A, dejaron constancia de cada diseño con su contexto, y los recorridos con el APK instalado en un teléfono Android permitieron revisar las pantallas de la aplicación.

Tabla 4. *Instrumentos de recolección de datos de la aplicación móvil*

Instrumento	**Qué mide**	**Cómo se aplica**
Historial del repositorio	Avance por semana y por carpeta	Conteo de cambios registrados
Decisiones numeradas	Contexto, decisión y consecuencias de cada diseño	Lectura de la carpeta docs/decisiones (Anexo A)
Recorridos en teléfono	Pantallas de la aplicación	Recorrido de cada pantalla con el APK instalado

4.1.3. RECURSOS UTILIZADOS

En lo humano, el desarrollo de los módulos 1 y 2 y de la carpeta compartida se coordinó con los equipos de los otros módulos y con el servidor central. Las herramientas de desarrollo se enlistan en la Tabla 2. A ellas se suman la biblioteca Argon2kt 1.5.0, para guardar contraseñas, Navigation Compose 2.8.2, para la navegación entre pantallas, Lifecycle 2.8.6 y Coroutines 1.9.0, todas declaradas en el archivo de versiones del proyecto.

En lo económico, la decisión 0008 estableció que la aplicación se distribuye solo como APK desde la página del proyecto y que todo lo que cueste dinero se evita salvo que sea indispensable. Por eso las herramientas de esta parte son gratuitas y de código abierto: Android Studio, Kotlin, Jetpack Compose, Room, Hilt, DataStore y GitHub. El teléfono de pruebas y el tiempo de desarrollo son los únicos recursos que no se compraron.

4.1.4. CRONOGRAMA DE ACTIVIDADES REALIZADAS

La Tabla 5 ordena las actividades centrales con su fecha. Cada una puede comprobarse en el historial de cambios o en las decisiones numeradas, y las fechas corresponden a los commits del repositorio en las carpetas de los módulos 1 y 2, el motor de voz y el servicio de escucha.

Tabla 5. *Actividades centrales del desarrollo de la aplicación móvil*

Fecha (2026)	**Actividad**	**Evidencia**
13 de julio	Primer cambio del repositorio (prototipo previo a la estadía)	Historial
18 de septiembre	Corrección de la compilación y entorno virtual funcional	Historial
25 de septiembre	Interfaz para personas invidentes: menú, voz global, personalización, notificaciones y widget; decisiones 0006 y 0007	Historial y decisiones
26 y 27 de septiembre	Pantalla negra, brillo mínimo, módulo Chats y comandos con tolerancia a errores de reconocimiento	Historial y decisión 0003
28 de septiembre	IA apagada por defecto, borrar la cuenta y los datos, y correcciones de privacidad y accesibilidad táctil	Historial y decisión 0002
30 de septiembre	Roles (administrador, directivo, educador, alumno), primer panel de Gestión y sesión recordada	Historial
1 al 3 de octubre	Validación de cuentas, editar cuenta, soporte y tamaños de tarjeta unificados	Historial
8 de octubre	Decisiones 0008 (APK sin tiendas y costo cero), 0011 y 0012 (tamaños y micrófono fijo)	Decisiones
27 de octubre	Contraseña de 15 letras normalizada, correo opcional y avisos de servidor dormido, límite y falla	Historial

Fecha (2026)	Actividad	Evidencia
30 de octubre	Pantalla dividida con micrófono al 42 %, tarjeta iluminada, confirmación de 18 años o más y base de datos versión 19	Historial
2 de noviembre	Interfaz ServicioDeCuentas, contraseñas con Argon2id, widgets de un solo botón e ícono de la aplicación	Historial

4.2. ANÁLISIS DE REQUISITOS

Los requisitos salen de los objetivos específicos, de las decisiones documentadas y de las necesidades del CECAPI descritas en el capítulo II. Los funcionales de la Tabla 6 describen lo que la aplicación debe hacer y cada uno indica el objetivo al que responde; los no funcionales de la Tabla 7 describen las condiciones en las que debe hacerlo y cómo se verifica cada uno.

Tabla 6. *Requisitos funcionales de la aplicación móvil*

Clave	Requisito	Objetivo
RF1	Recorrer todas las pantallas con lector de pantalla, con etiquetas descriptivas y señales distintas del color	O1
RF2	Operar la aplicación con comandos de voz globales y propios de cada pantalla	O2
RF3	Activar la escucha con una palabra de activación (el nombre del asistente y, si se activa, hola) y detenerla con voz	O2
RF4	Enviar a la IA las frases que la aplicación no entiende y leer su respuesta en voz alta	O2
RF5	Avisar por voz cuando no se entendió una frase o cuando el servidor tarda o falla	O2
RF6	Crear una cuenta con usuario, contraseña y confirmación de 18 años o más, e iniciar y cerrar sesión	O3
RF7	Validar el correo con un código y recuperar la contraseña con el servidor central	O3
RF8	Instalarse como APK en teléfonos con Android 8.0 o posterior	O4

Tabla 7. *Requisitos no funcionales de la aplicación móvil*

Clave	Requisito	Cómo se verifica
RNF1	Accesibilidad: todo operable con voz, con lector de pantalla y sin depender del color	Revisión del código y recorridos en teléfono
RNF2	Privacidad: la IA y la escucha en segundo plano empiezan apagadas y la persona puede borrar su cuenta y sus datos	Revisión de código y decisiones 0002 y 0007
RNF3	Seguridad: contraseñas con Argon2id, bloqueo tras intentos fallidos y sin copias de seguridad de la app	Revisión de código y recorridos en teléfono
RNF4	Costo directo cero en la aplicación móvil y en sus herramientas de desarrollo	Decisión 0008

Clave	Requisito	Cómo se verifica
RNF5	Compatibilidad: nivel de API 26 como mínimo y 35 como objetivo	Archivo build.gradle.kts
RNF6	Mantenibilidad: una carpeta por módulo, versiones congeladas y decisiones registradas	Revisión de la estructura del repositorio

4.3. DISEÑO DE LA SOLUCIÓN

4.3.1. ARQUITECTURA DE LA APLICACIÓN

La aplicación sigue una arquitectura por capas, representada en la Figura 2. Las pantallas se escriben con Jetpack Compose y solo muestran información y reciben toques o voz; cada pantalla tiene un ViewModel que decide qué hacer; el núcleo compartido reúne el motor de voz, la base de datos y las utilerías de seguridad; y debajo están los servicios del teléfono, como el reconocimiento de voz, la síntesis de voz y el almacenamiento local. Hilt entrega a cada clase los objetos que necesita en lugar de que los cree por su cuenta.

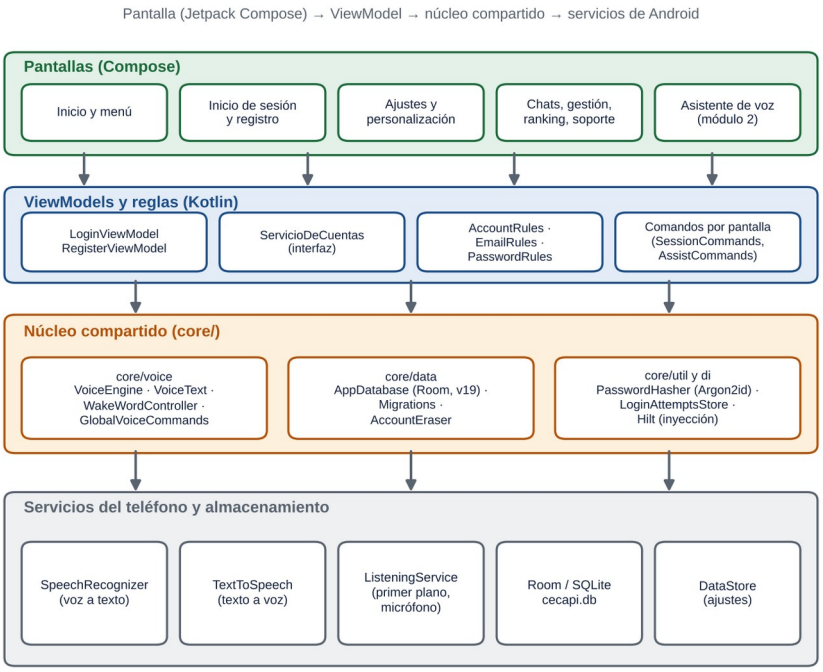


Figura 2. *Arquitectura por capas de la aplicación Android*

4.3.2. NAVEGACIÓN Y PANTALLAS

La aplicación tiene 17 destinos de navegación, de los que la Figura 3 muestra los de los módulos 1 y 2 y enlaza los de otros equipos. Antes de iniciar sesión se pueden usar el inicio, el inicio de sesión y el registro; algunas pantallas, como Chats, Gestión, Clasificación y Editar cuenta, piden una cuenta con sesión iniciada. El acceso se maneja con roles, no con planes (decisión 0009), y la Tabla 8 resume qué puede hacer cada rol.

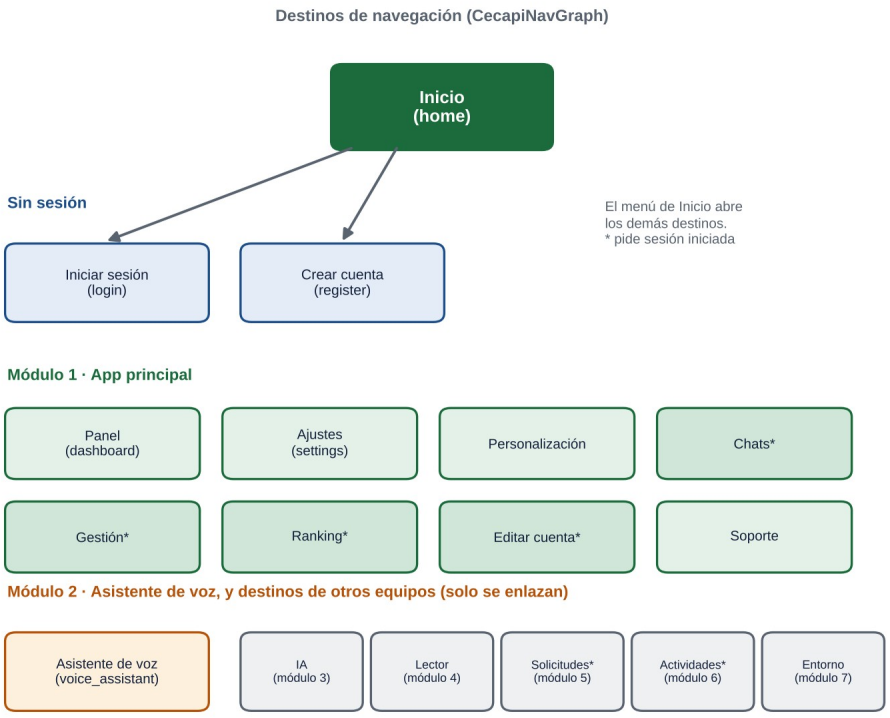


Figura 3. *Destinos de navegación de la aplicación*

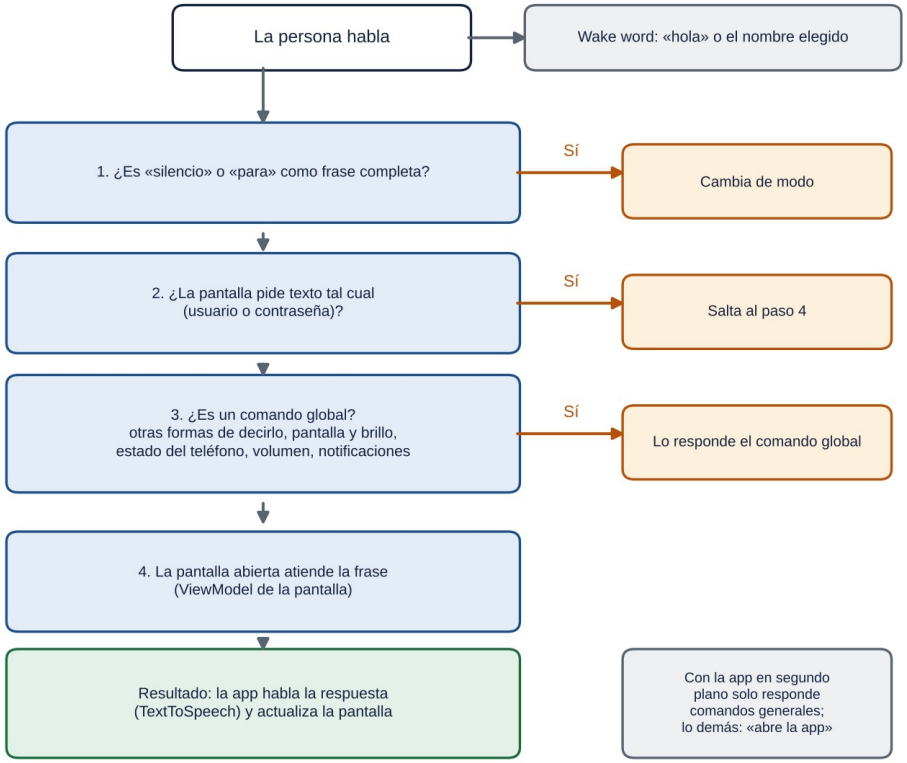
Tabla 8. *Roles de las cuentas y lo que pueden hacer*

Rol	A quién ve	Qué puede hacer
Administrador	Todas las instituciones y los usuarios independientes	Cambiar roles, asignar educador y validar a cualquiera
Directivo	Toda su institución	Cambiar roles sin dar administrador, asignar educador y validar educadores de su institución
Educador	Solo a sus alumnos	Validar a sus alumnos
Alumno	A nadie	Ver quién es su educador y reportar un problema

Las decisiones de diseño de la interfaz responden a la necesidad de quien no ve la pantalla. Los botones y tarjetas siguen tamaños definidos en un componente compartido, de modo que ninguna pantalla dibuja botones propios (decisión 0012); el micrófono ocupa siempre el mismo lugar, para que se encuentre sin buscarlo; la pantalla puede ponerse en negro, con salida por voz y por toque (decisión 0003); y la tarjeta seleccionada se ilumina con el color de su módulo para indicar dónde está la persona.

4.3.3. FLUJO DE UNA FRASE DE VOZ

Toda la aplicación usa un solo motor de voz, porque Android solo permite un reconocedor y una síntesis de voz confiables por proceso. Ninguna pantalla crea el suyo: reciben las frases que el motor les pasa. La Figura 4 muestra el orden en que se atiende una frase. Primero se revisa si es una orden de control, como silencio o para; luego, si la pantalla pide el texto tal cual, como el usuario o la contraseña, se salta a la pantalla; si no, se prueba con los comandos globales y, por último, con los de la pantalla abierta.



Fuente: docs/explicacion/voz.md y core/voice/VoiceEngine.kt. «Silencio» y «para» no necesitan el wake word.

Figura 4. *Recorrido de una frase dicha al asistente*

El asistente tiene tres modos, que muestra la Figura 5. En el modo activo habla y escucha; en silencio calla, pero sigue atento a la palabra de activación; y detenido no hace nada hasta que la persona abre la aplicación de nuevo o pulsa el botón Detener del aviso fijo. La decisión 0006 fijó que las órdenes silencio y para solo valen como frase completa, porque la palabra para también aparece en frases comunes como para qué sirve. Las frases de cada orden se listan en la Tabla 9.

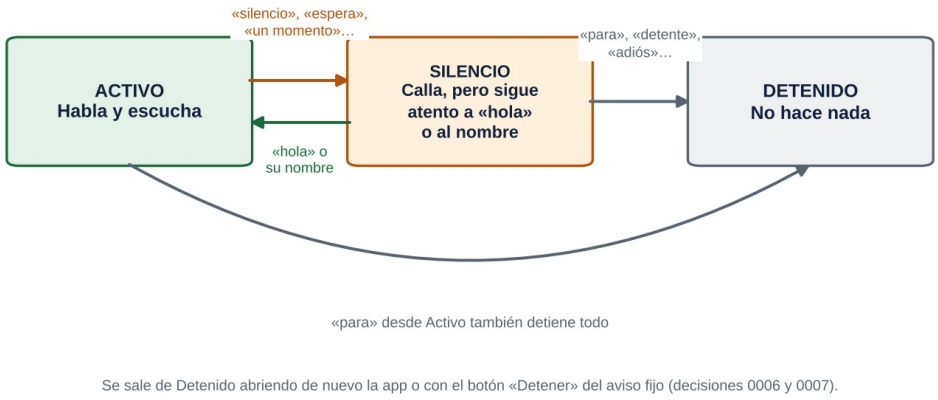


Figura 5. *Modos del asistente: activo, silencio y detenido*

Tabla 9. *Órdenes de control del asistente (decisión 0006)*

Orden	Frases que la activan	Qué hace
Silencio	silencio, espera, espérame, cállate, un momento, un segundo	Calla al asistente; vuelve con hola o con su nombre
Para	para, detente, alto, basta, para ya, ya para, hasta luego, adiós, apágate	Detiene todo hasta abrir de nuevo la aplicación

4.3.4. MODELO DE DATOS

La aplicación guarda sus datos en una sola base local, cecapi.db, con Room en la versión 19 del esquema y 22 entidades en total. La Figura 6 muestra las tablas de los módulos 1 y 2: usuarios, con el hash de la contraseña, el rol y la fecha en que la persona confirmó tener 18 años o más; configuración y permisos por módulo de cada usuario; y las tablas de comandos de voz, su historial y sus respuestas auditivas. El módulo 1 incluye además cambios de cuenta e incidencias de soporte, y la pantalla Chats lee la tabla de consultas del módulo 3.

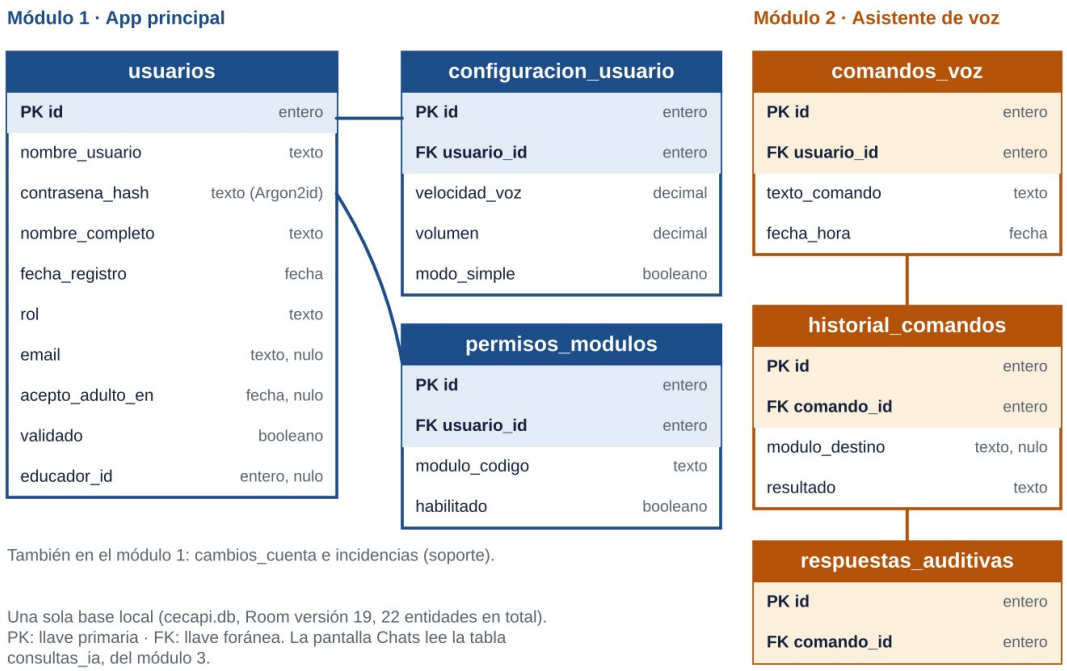


Figura 6. *Tablas de los módulos 1 y 2 en Room*

4.3.5. CUENTAS Y CONTRASEÑAS

El diseño de las cuentas se decidió el 30 de octubre de 2026: una sola forma de ingreso, con una cuenta propia, sin acceso con Google, y un código al correo solo para registrarse y recuperar la contraseña. La Figura 7 distingue la cuenta guardada en el teléfono, que es la que funciona en la aplicación, de la cuenta en el servidor con código por correo, que es la siguiente etapa. La interfaz ServicioDeCuentas separa las pantallas de la forma de guardar las cuentas, de modo que el cambio al servidor no obliga a tocar la interfaz.

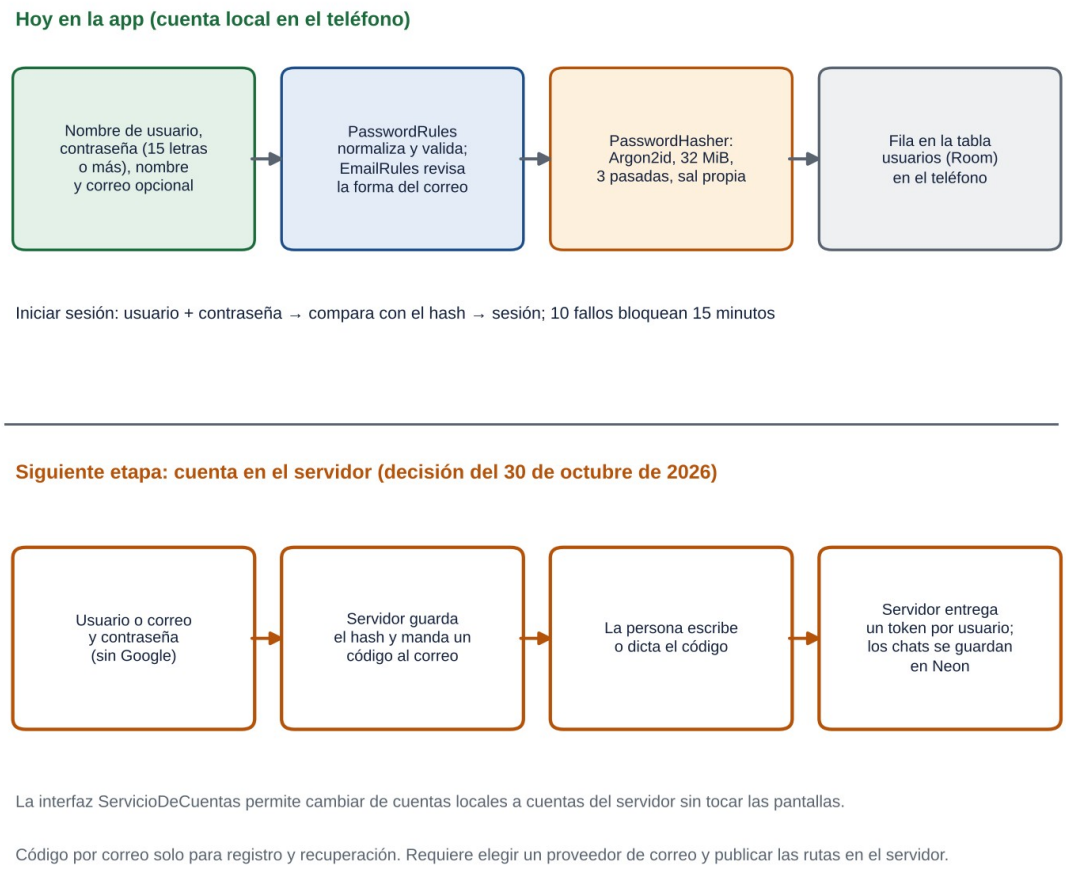


Figura 7. *Cuenta local actual y flujo previsto con código por correo*

4.4. IMPLEMENTACIÓN POR OBJETIVO

4.4.1. INTERFAZ ACCESIBLE (O1)

La interfaz se programó con Jetpack Compose y la guía de accesibilidad de Android. En el módulo 1 el código declara 15 descripciones de contenido y 22 usos de semántica, entre ellos encabezados, roles, descripciones de estado y regiones de lectura. Además, los recuadros de carga muestran una rueda con mensaje que TalkBack anuncia, hay un botón visible para volver y la barra de desplazamiento se unificó en los siete paneles.

La pantalla dividida, creada el 30 de octubre, reparte el espacio con una proporción fija: el contenido ocupa el 58 % superior y el micrófono el 42 % inferior, y toda esa zona es el botón. Un toque habla, dos toques o un toque con dos dedos callan al asistente y mantener oprimido da la ayuda. Antes el micrófono era un panel flotante con medidas fijas que no se adaptaban al

tamaño del teléfono ni al texto grande, y el contenido seguía existiendo debajo, donde el lector de pantalla podía enfocar elementos tapados.

4.4.2. ASISTENTE DE VOZ (O2)

El asistente envuelve SpeechRecognizer y TextToSpeech, los servicios de reconocimiento y de síntesis de voz de Android, en una sola clase llamada VoiceEngine. El asistente atiende su nombre, que la persona puede personalizar, y la palabra hola cuando esa opción está activada, y un controlador mantiene la escucha encendida mientras al menos una pantalla la necesita. La voz se ejecuta fuera del hilo principal, lo que evita que la aplicación se congele si el audio del teléfono se traba, y existe un botón para elegir el motor de voz cuando el teléfono no tiene uno activo.

La clase VoiceText quita acentos y mayúsculas y compara por palabras completas, de modo que menudo no se confunde con menú. Tolera una letra mal en palabras largas y una palabra partida en dos, como en isquierda por izquierda o en frente por enfrente, para compensar los errores del reconocimiento. Los comandos globales funcionan en cualquier pantalla e incluyen el estado del teléfono, el volumen, las notificaciones, la pantalla negra, la velocidad de la voz y la ayuda para aprender a usar la aplicación.

La escucha fuera de la aplicación es opcional y empieza apagada (decisión 0007). Cuando se activa en Configuración funciona como un servicio en primer plano con tipo micrófono, con un aviso fijo y un botón Detener, y fuera de la aplicación solo responde a los comandos generales. Solo existe con una sesión iniciada: se enciende al entrar y se apaga al cerrar sesión, avisando por voz. Android exige declarar un tipo para cada servicio en primer plano desde el nivel de API 34, y para el micrófono se requieren los permisos correspondientes (Android Developers, 2026d).

Las preguntas que la aplicación no entiende pueden pasar a la inteligencia artificial del servidor central mediante un resolvedor registrado al abrir la aplicación. Por la decisión 0002, la IA empieza apagada y se activa en Configuración, en la sección Privacidad; mientras esté apagada ninguna frase sale del teléfono hacia el servidor de IA. Las preguntas de información general, como qué es o quién es, se consultan además en Wikipedia y Wikidata, servicios de terceros. El servidor y el módulo de consultas los desarrollan otros equipos y se describen en sus memorias.

4.4.3. REGISTRO E INICIO DE SESIÓN (O3)

Las cuentas de la aplicación se guardan en el teléfono mediante la clase CuentasLocales, que implementa la interfaz ServicioDeCuentas. La contraseña de CECAPI se diseñó para dictarse: debe tener al menos 15 letras, solo letras de la a a la z y la ñ, y antes de validarla se normaliza quitando espacios, acentos y mayúsculas, de modo que una frase como Perro Café Rojo, dicha o escrita, llega igual. Una frase larga es fácil de decir y difícil de adivinar, y no depende de números ni de símbolos que el reconocimiento escribe de formas distintas.

La contraseña se guarda con Argon2id, con los parámetros de la Tabla 10, y cada cuenta lleva su propia sal. Tras diez intentos fallidos el inicio de sesión se bloquea 15 minutos, medidos con un reloj interno que la persona no puede cambiar. El correo es opcional y se revisa en el teléfono por su forma y por errores comunes de escritura.

Tabla 10. *Parámetros de la protección de contraseñas en la aplicación*

Parámetro	**Valor**
Algoritmo	Argon2id con sal propia por cuenta
Memoria	32 MiB
Pasadas	3
Hilos	1
Largo del resumen	32 bytes
Largo de la sal	16 bytes
Bloqueo por intentos	10 fallos, 15 minutos

Al registrarse se pide confirmar que se tiene 18 años o más y se guarda en qué momento lo declaró la persona, con la migración a la versión 19 de la base. Esta confirmación es declarativa y no sustituye una verificación de edad. El código de seis dígitos enviado al correo, la recuperación de la contraseña y las cuentas en el servidor forman la siguiente etapa, porque requieren que el servidor central publique las rutas de cuenta y que se elija un proveedor de correo.

4.4.4. DISTRIBUCIÓN COMO APK (O4)

La decisión 0008 estableció que en Android el APK se distribuirá solo desde la página de descargas del proyecto, con su huella SHA-256 y un guion de instalación por voz. El archivo de compilación de la aplicación activa para la versión de lanzamiento la reducción y la ofuscación

del código, que lo hacen más ligero y más difícil de descompilar. La versión de la aplicación es 1.0, con nivel de API 26 como mínimo y 35 como objetivo.

Durante el desarrollo, el APK se compila y se instala directamente en el teléfono, que es la modalidad de instalación directa que persigue el objetivo. La firma digital con una clave propia, que Android exige para instalar o actualizar un APK (Android Developers, 2026k), y la publicación de cada versión como release en GitHub con su huella SHA-256 forman la siguiente etapa, que convierte esta compilación en una distribución para los alumnos del CECAPI. La decisión 0008 advierte además que, desde Android 13, un APK descargado de una página no puede activar la accesibilidad hasta que la persona permite la configuración restringida, por lo que el guion de instalación debe explicarlo.

4.5. SEGURIDAD Y PRIVACIDAD DE LA APLICACIÓN

La Tabla 11 reúne los controles aplicados en la aplicación. El principio de trabajo fue que lo que no se necesita no se guarda y que la persona decide qué sale de su teléfono. Las dos funciones que más datos pueden mover, la IA y la escucha en segundo plano, empiezan apagadas.

Tabla 11. *Controles de seguridad y privacidad aplicados en la aplicación móvil*

Control	**Qué evita**	**Fuente**
IA apagada por defecto	Enviar frases al servidor de IA sin que la persona lo decida	Decisión 0002
Escucha fuera de la app apagada por defecto	Escuchar sin que la persona lo sepa	Decisión 0007
Contraseñas con Argon2id y sal propia	Que una base filtrada revele las contraseñas	Código de la aplicación
Bloqueo tras 10 intentos fallidos	Adivinar contraseñas probando	Código de la aplicación
Sin copias de seguridad de la app	Que los datos salgan por la copia automática de Android	Manifiesto (allowBackup apagado)
Tráfico sin cifrar solo en la versión de depuración	Enviar datos por HTTP en la versión de lanzamiento	Archivo build.gradle.kts
Borrar mi cuenta y mis datos	Conservar datos de quien ya no usa la app	Código de la aplicación
Confirmación de 18 años o más	Uso por menores sin declaración	Registro de cuenta

Sobre el micrófono, la documentación de Android advierte que el reconocimiento de voz probablemente transmite el audio a servidores remotos (Android Open Source Project [AOSP], s. f.-b), de modo que lo que se dice cerca del teléfono con la escucha activa puede salir del

dispositivo, también el de personas cercanas que no usan la aplicación. Por eso la escucha fuera de la app es opcional, muestra un aviso fijo mientras funciona y solo puede iniciarse con una sesión abierta.

4.6. PRUEBAS REALIZADAS

Las pruebas se realizaron de forma manual con el APK instalado en un teléfono Android. Cada versión se compiló, se instaló y se recorrieron sus pantallas, y los fallos encontrados se corrigieron en la versión siguiente. Este método permitió revisar la aplicación sobre el dispositivo en el que se usa, con su motor de voz, su pantalla y sus permisos reales.

El reconocimiento de voz depende de que el teléfono tenga descargado el paquete de voz del idioma, y las funciones disponibles dependen de la versión de Android de cada aparato. El nivel de API 26 es el mínimo con el que se compila la aplicación, de modo que los teléfonos con Android 8.0 o posterior pueden instalarla.

CAPÍTULO V
RESULTADOS Y CONCLUSIONES

5.1. RESULTADOS

Los resultados se presentan en el orden de los objetivos específicos del capítulo II. La Tabla 12 y la Figura 8 resumen el alcance logrado en cada objetivo al 2 de noviembre de 2026, según el código del repositorio y los recorridos de prueba en el teléfono.

Tabla 12. *Alcance de los objetivos específicos al 2 de noviembre de 2026*

Objetivo	**Alcance logrado**	**Siguiente etapa**
O1. Pantallas accesibles	17 destinos de navegación, micrófono fijo, tamaños de botón unificados, pantalla dividida y etiquetas descriptivas en el módulo 1	Extender las etiquetas a todas las pantallas
O2. Asistente de voz	Motor de voz único, palabra de activación, tolerancia a errores, comandos globales y por pantalla, escucha en segundo plano opcional y preguntas a la IA conectadas en el código	Comprobar la conexión con el servidor central
O3. Registro e inicio de sesión	Cuenta local con contraseña de 15 letras, Argon2id, bloqueo por intentos y confirmación de 18 años	Código por correo y cuentas en el servidor
O4. Distribución como APK	APK de instalación directa compilado e instalado en teléfono, con versión de lanzamiento optimizada	Firma digital y publicación en GitHub Releases

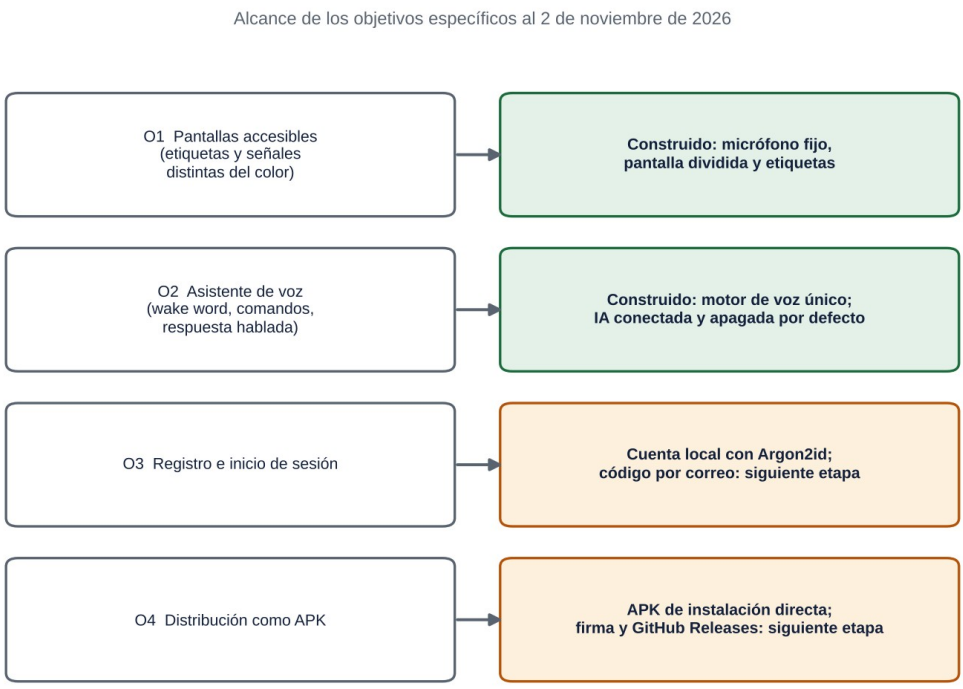


Figura 8. *Alcance de los objetivos específicos al 2 de noviembre de 2026*

5.1.1. INTERFAZ ACCESIBLE (O1)

La aplicación tiene 17 destinos de navegación, 11 de ellos del módulo 1, y todos comparten el micrófono fijo en la zona inferior y los tamaños de botón de la decisión 0012. El código del módulo 1 declara 15 descripciones de contenido y 22 usos de semántica, y los recuadros de carga anuncian su mensaje con el lector de pantalla. El APK instalado en un teléfono Android se usó durante el desarrollo para recorrer estas pantallas.

5.1.2. ASISTENTE DE VOZ (O2)

El asistente reconoce comandos globales y propios de cada pantalla, con tolerancia a acentos, a una letra mal y a palabras partidas, y se detiene o calla con las órdenes de la decisión 0006. Funciona con un solo motor de reconocimiento y síntesis de voz para toda la aplicación, atiende la palabra de activación y ofrece la escucha en segundo plano como opción apagada por defecto, que se activa en Configuración.

5.1.3. REGISTRO E INICIO DE SESIÓN (O3)

El registro y el inicio de sesión funcionan con cuentas guardadas en el teléfono, con cuatro roles y con la confirmación de 18 años o más. La contraseña de al menos 15 letras se normaliza antes de validarse, se protege con Argon2id y el acceso se bloquea 15 minutos tras diez intentos fallidos. La interfaz ServicioDeCuentas deja preparada la conexión con las cuentas del servidor, a las que se sumarán el código por correo y la recuperación de la contraseña.

5.1.4. DISTRIBUCIÓN COMO APK (O4)

La aplicación se compila como APK de instalación directa, con la versión de lanzamiento reducida y ofuscada, y se instala en teléfonos Android con nivel de API 26 o superior. La decisión 0008 fijó el canal de distribución: el APK se entrega desde la página del proyecto con su huella SHA-256 y un guion de instalación por voz, sin tiendas de aplicaciones y con costo cero. La firma con clave propia y la publicación de cada versión en GitHub Releases completan esa distribución.

5.2. CONCLUSIONES

El proyecto produjo una aplicación Android de asistencia por voz con una interfaz pensada para personas ciegas o con baja visión, un motor de voz único y cuentas locales con contraseñas protegidas, y dejó documentadas las decisiones que la sustentan. Las conclusiones siguientes se organizan por objetivo específico y distinguen lo cumplido de lo que queda como siguiente etapa.

El primer objetivo se cumplió en parte. Se diseñaron las pantallas con navegación consistente, micrófono fijo, tamaños de botón unificados, pantalla dividida y tarjeta iluminada por módulo, y con etiquetas descriptivas para el lector de pantalla en las pantallas del módulo 1. Extender esas etiquetas a las demás pantallas es el trabajo siguiente.

El segundo objetivo se cumplió en parte: se programó en el teléfono un asistente que usa un solo motor de reconocimiento y síntesis de voz, atiende una palabra de activación, tolera errores del reconocimiento y reconoce comandos globales y propios de cada pantalla. La función de enviar preguntas a la inteligencia artificial quedó conectada en el código y apagada por defecto por privacidad, y su funcionamiento completo depende del servidor central, que desarrolla otro equipo.

El tercer objetivo se cumplió en parte: existe el registro y el inicio de sesión con una cuenta propia y una contraseña de al menos 15 letras pensada para dictarse, protegida con Argon2id, con bloqueo por intentos y confirmación de mayoría de edad. La validación del correo con un código y la recuperación de la contraseña dependen del servidor central y quedan como siguiente etapa, y la interfaz ServicioDeCuentas permite incorporarlas sin rehacer las pantallas.

El cuarto objetivo está en proceso: la aplicación se compila como APK de instalación directa y se instala en el teléfono, y la decisión 0008 fijó que se distribuirá sin tiendas de aplicaciones y con costo cero. La firma digital propia y la publicación en GitHub Releases son el paso siguiente. En conjunto, la aplicación reúne comandos de voz globales y por pantalla, respuestas habladas, cuentas locales y ajustes de voz, y deja definida la ruta para completar las cuentas con el servidor y la distribución del APK.

5.3. RECOMENDACIONES

Las recomendaciones se dirigen a quien continúe el proyecto y siguen el orden de importancia. La primera es cerrar la distribución, porque con un APK firmado y publicado los alumnos del CECAPI podrán instalar y usar lo que se desarrolló, y las demás recomendaciones se apoyan en esa entrega.

- **Firmar y publicar el APK.** Crear una clave de firma propia, guardarla fuera del repositorio, automatizar la compilación y la publicación en GitHub Releases con su huella SHA-256 y escribir el guion de instalación por voz.
- **Conectar las cuentas al servidor.** Implementar una segunda versión de ServicioDeCuentas que llame a las rutas del servidor, con el código por correo para registro y recuperación de la contraseña.
- **Agregar el aviso de privacidad y los términos a la aplicación.** Leerlos por partes y por audio, con aceptación registrada.
- **Evaluar el uso con alumnos del CECAPI.** Medir el tiempo y los pasos de tareas cotidianas con el lector de pantalla y con la aplicación, para valorar con datos cuánto reduce el asistente de voz.

Estas acciones llevarían la aplicación de un desarrollo funcional a una herramienta instalada y evaluada por las personas que la van a usar, que es la condición necesaria para recomendar su adopción por el CECAPI y para que el asistente de voz se convierta en una ayuda cotidiana de sus alumnos.

REFERENCIAS BIBLIOGRÁFICAS

Android Developers. (2026a). *Build accessible apps*. Recuperado el 27 de octubre de 2026, de https://developer.android.com/guide/topics/ui/accessibility

Android Developers. (2026b). *DataStore*. Recuperado el 27 de octubre de 2026, de https://developer.android.com/topic/libraries/architecture/datastore

Android Developers. (2026c). *Dependency injection with Hilt*. Recuperado el 27 de octubre de 2026, de https://developer.android.com/training/dependency-injection/hilt-android

Android Developers. (2026d). *Foreground service types*. Recuperado el 27 de octubre de 2026, de https://developer.android.com/develop/background-work/services/fgs/service-types

Android Developers. (2026e). *Get started with Jetpack Compose*. Recuperado el 27 de octubre de 2026, de https://developer.android.com/develop/ui/compose/documentation

Android Developers. (2026f). *Kotlin and Android*. Recuperado el 27 de octubre de 2026, de https://developer.android.com/kotlin

Android Developers. (2026g). *Network security configuration*. Recuperado el 27 de octubre de 2026, de https://developer.android.com/privacy-and-security/security-config

Android Developers. (2026h). *Principles for improving app accessibility*. Recuperado el 27 de octubre de 2026, de https://developer.android.com/guide/topics/ui/accessibility/principles

Android Developers. (2026i). *Register for limited distribution on Android devices*. Recuperado el 27 de octubre de 2026, de https://developer.android.com/developer-verification/guides/limited-distribution

Android Developers. (2026j). *Save data in a local database using Room*. Recuperado el 27 de octubre de 2026, de https://developer.android.com/training/data-storage/room

Android Developers. (2026k). *Sign your app*. Recuperado el 27 de octubre de 2026, de https://developer.android.com/studio/publish/app-signing

Android Open Source Project. (s. f.-a). *RecognizerIntent.java [Código fuente]*. Recuperado el 27 de octubre de 2026, de https://android.googlesource.com/platform/frameworks/base/+/refs/heads/main/core/java/android/speech/RecognizerIntent.java

Android Open Source Project. (s. f.-b). *SpeechRecognizer.java [Código fuente]*. Recuperado el 27 de octubre de 2026, de https://android.googlesource.com/platform/frameworks/base/+/refs/heads/main/core/java/android/speech/SpeechRecognizer.java

Android Open Source Project. (s. f.-c). *TextToSpeech.java [Código fuente]*. Recuperado el 27 de octubre de 2026, de https://android.googlesource.com/platform/frameworks/base/+/refs/heads/main/core/java/android/speech/tts/TextToSpeech.java

Forsythe, M. (2026, 18 de junio). *Android developer verification: Building a safer ecosystem together*. Android Developers Blog. Recuperado el 27 de octubre de 2026, de https://android-developers.googleblog.com/2026/06/android-developer-verification.html

GitHub. (2026). *About releases*. GitHub Docs. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/repositories/releasing-projects-on-github/about-releases

Google. (s. f.-a). *Get started with the Gemini mobile app - Android phone & tablet*. Gemini Apps Help. Recuperado el 29 de octubre de 2026, de https://support.google.com/gemini/answer/14554984

Google. (s. f.-b). *Get started with Voice Access spoken commands*. Android Accessibility Help. Recuperado el 27 de octubre de 2026, de https://support.google.com/accessibility/android/answer/6151848

Google. (2026, 28 de abril). *Gemini API Additional Terms of Service*. Google AI for Developers. Recuperado el 29 de octubre de 2026, de https://ai.google.dev/gemini-api/terms

Groq. (s. f.). *Your data*. GroqCloud Docs. Recuperado el 29 de octubre de 2026, de https://console.groq.com/docs/your-data

Instituto Nacional de Estadística y Geografía. (2024, 28 de noviembre). *Estadísticas a propósito del Día Internacional de las Personas con Discapacidad (3 de diciembre)* (Comunicado de prensa núm. 684/24). Recuperado el 27 de octubre de 2026, de https://www.inegi.org.mx/contenidos/saladeprensa/aproposito/2024/EAP_PCD24.pdf

Ley Federal de Protección de Datos Personales en Posesión de los Particulares. (2025, 20 de marzo). Diario Oficial de la Federación. Recuperado el 29 de octubre de 2026, de https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf

National Institute of Standards and Technology. (2025). *NIST special publication 800-63B*. Recuperado el 26 de octubre de 2026, de https://pages.nist.gov/800-63-4/sp800-63b.html

Neon. (s. f.). *Data Processing Addendum*. Recuperado el 29 de octubre de 2026, de https://neon.com/pdf/DPA.pdf

OWASP Cheat Sheet Series. (s. f.-a). *Authentication*. Recuperado el 27 de octubre de 2026, de https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP Cheat Sheet Series. (s. f.-b). *Password Storage*. Recuperado el 29 de octubre de 2026, de https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Render. (s. f.-a). *Data Processing Addendum*. Recuperado el 29 de octubre de 2026, de https://render.com/dpa

Render. (s. f.-b). *Terms of Service*. Recuperado el 29 de octubre de 2026, de https://render.com/terms

Universidad Tecnológica de Durango. (s. f.). *Manual para la estructura y contenido de la memoria de estadía para técnico superior universitario, licenciaturas e ingenierías*.

World Wide Web Consortium. (2024). *Web Content Accessibility Guidelines (WCAG) 2.2*. Recuperado el 27 de octubre de 2026, de https://www.w3.org/TR/WCAG22/

ANEXOS

El material de apoyo de esta memoria son las decisiones de diseño citadas en esta memoria, con lo que se decidió y su consecuencia, y se presentan en el Anexo A.

Anexo A. *Decisiones de diseño citadas en esta memoria*

Núm.	Decisión	Qué se decidió	Consecuencia
0001	Versiones de Gradle, AGP y Kotlin congeladas	AGP 8.5.2, Gradle 8.9, Kotlin 2.0.21, KSP 2.0.21-1.0.28, Room 2.6.1 y Hilt 2.51.1, con compileSdk y targetSdk 35 y minSdk 26; no se cambian sin acuerdo.	Una actualización a versiones más nuevas había roto la compilación desde un clon limpio.
0002	IA como cerebro temporal, apagada por defecto	La IA solo traduce una frase no entendida a un comando conocido o da una respuesta corta. Empieza apagada y se activa en Configuración, sección Privacidad.	Mientras está apagada, ninguna frase se envía a un servidor de IA.
0003	Pantalla negra con salida por voz y por toque	Pantalla negra y brillo mínimo funcionan en cualquier pantalla, con dos salidas: decir pantalla normal y mantener presionada la pantalla.	Los comandos de pantalla se atienden antes que cualquier otro, para que la salida funcione siempre.
0005	Responsable legal, menores y datos reales	El responsable legal es José Fernández Quintana, a título personal, mientras solo haya cuentas de demostración; antes de usar datos reales se buscará un convenio con el CECAPI. El aviso de privacidad y el consentimiento dentro de la app esperan la revisión de una persona abogada.	Los textos legales son borradores para revisión de abogado. Ya se construyó borrar la cuenta y los datos, la IA apagada por defecto y el inicio de sesión sin nombre de usuario en el registro.
0006	Silencio y para solo como frase completa	Las órdenes silencio y para solo valen como frase completa, porque para también aparece en frases comunes.	Se atienden antes que cualquier otro comando y funcionan sin decir la palabra de activación.
0007	Escuchar fuera de la app: opcional y apagado	La escucha fuera de la app se activa en Configuración, empieza apagada y funciona como servicio en primer plano con aviso fijo y botón Detener.	Fuera de la app solo responde a los comandos generales.
0008	Android: solo el APK desde la página del proyecto, y costo cero	El APK se distribuye solo desde la página de descargas, con su huella SHA-256 y un guion de instalación por voz. El único gasto aceptado es el dominio.	Desde Android 13, un APK descargado no puede activar la accesibilidad hasta permitir la configuración restringida; el guion debe explicarlo.
0009	El acceso se maneja con roles, no con planes	El acceso y lo que ve cada persona dependen solo de su rol, sin planes ni pagos dentro de la aplicación.	Las funciones se activan por rol, no por pago.

Núm.	Decisión	Qué se decidió	Consecuencia
0011	Versión mínima de Android	Mínimo de Android 5 (API 21) y recomendada Android 11 o superior; la aplicación avisa por voz qué funciones no existen en el teléfono.	El mínimo vigente en Gradle es API 26; bajarlo exige cambiar Gradle, que está congelado.
0012	Tamaños de botón y micrófono fijo	Seis tamaños de tarjeta en toda la app, cuatro de un componente compartido y dos del podio de la clasificación; el micrófono va anclado abajo, con la misma altura en todas las pantallas.	Ningún equipo hace botones a mano, y se revisan los tamaños y el micrófono en cada panel nuevo.
0018	Ubicación para orientarse: evaluación	Evaluación, sin implementar, de decir dónde está la persona. Identifica la discapacidad visual como dato sensible (ley federal de datos personales, art. 8).	La aplicación no usa ubicación, y se quitó un permiso de ubicación que no se usaba.